

# UML Sınıf Modelleri

## UML ile problemi modelleme

UML'in en popüler tarafı. Analiz modelini nasıl ifade edeceğinizi tanımlar.

Aslında en karmaşık olanı

- Modellemenin başında tüm detaylarını kullanmak zorunda değilsiniz.

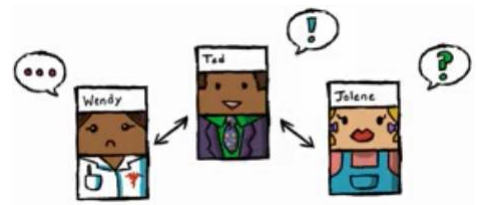
Bu derste tüm yönlerine bakacağız.

- İhtiyacınız olduğunda kullanabilmeniz için bilmeniz gerekli

UML Sınıf diyagramları Statik Yapı diyagramı olarak da bilinir

Sınıflar dışında arayüzler, nesneler, ilişkiler de yer alabilir.

UML Elkitabı bu konuda temel referans kaynağıdır.



## Sınıf

Benzer örneklerin kümesinin tanımıdır.

Neler sınıf olabilir?

- Etki alanı nesneleri
- Roller
- Olaylar
- Etkileşimler

İsimlerle aday sınıflar bulunabilir.

Genellikle üç parçalı dikdörtgen ile gösterilir.

- Ad kesindir, genellikle özellik ve işlemler de yer alır
- Sorumluluklar, aykırı durumlar, ... da ayrı parçalarda dörtgen içinde gösterilebilir.



## Ad Alanı

Sınıfın adı bir isim olmalıdır !!

Eğik yazılırsa (veya <<abstract>>) soyut sınıf anlamı taşır

- Soyut sınıf: alt sınıfların özelliklerini tanımlayan, kendilerinden asla örnek türetilmeyen kontratlardır.
- Ortak özellikleri olan alakalı sınıfları, bir soyut sınıf ile gruplayabilirsiniz.

Bunun dışında, stereotipler (kışeler) ile temel UML modelleme dilini genişletebilirsiniz.



## Nitelikler

Sınıfların nitelikleri vardır.

- Özellikler
- Davranışlar

Sınıflar gerçek hayatta var olurlar.

Nitelikler, aslında bilgisayardaki karşılıklarıdır.

- Özellikler: Örnek değişkenleri
- Davranışlar: Metotlar.

## Özellikler Alanı

Tüm özelliklerin adları, tipleri, ve erişim türleri vardır.

Erişim türleri görünürlüklerini (diğer sınıflardan erişilebilirliklerini) tanımlar ve başta opsiyoneldir, sonra ekleyebilirsiniz.

+: Genel görünür.

-: Özel görünür

#: Korumalı görünür (sadece alt sınıflar)

~: Paketlerde, paket içinde görünür.

Çoğullamalar da belirtilir.

Özelliklerin tipleri tanımlanmalıdır.

İlk değer atamaları istenirse yapılabilir. Türetildiği (hesaplanarak bulunduğu, atanmadığı) da belirtilebilir.

Özel tanımlamalar {...} ile yapılabilir.

- {frozen}: özelliğin değeri değiştirilemez.

## İşlemler Alanı

İsimleri tanımlanmalıdır.

Metotların görünürlüğü özelliklerdeki gibi tanımlanabilir.

Değer döndüren işlemlerin dönüş tipi belirlenmelidir.

Varsa Parametreleri, tipleri tanımlanmalıdır.

- istenirse ilk değer atamaları ve içeri/dışarı tanımları yapılabilir.

Özel tanımlamalar {...} ile yapılabilir.

- {query}: bu bazı özelliklerin bilgisi veren sorgu işlemidir,
- {concurrency}
- {abstract}

Sınıf Kapsamı bu işlem sınıf kapsamındadır, tekil nesnelere ait değildir. Ör. Bu sınıftan kaç nesne türettik?

## Örnek Sınıflar

| Rectangle                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pt:Point<br>p2:Point                                                                                                                                                                          |
| <pre> &lt;&lt;constructor&gt;&gt; Rectangle(pt:Point, p2:Point) &lt;&lt;query&gt;&gt; area(): Real aspect(): Real ... &lt;&lt;update&gt;&gt; move(delta: Point) scale(ratio: Real) ... </pre> |

| Reservation                                                          |
|----------------------------------------------------------------------|
| <pre> operations guarantee() cancel() change(newDate: Date) </pre>   |
| <pre> responsibilities bill no-shows match to available rooms </pre> |
| <pre> exceptions invalid credit card </pre>                          |

## İleri Sınıf Modeli

---

### Arayüzler

- Bir arayüz yaratıcısı ile programınızda tip tanımlayabilirsiniz.
- Arayüz sisteme neler sunuyor, sistemden neler bekliyor

### Parametrik sınıflar

- Java generics ya da C++ templates olarak gerçekleşir. Koleksiyonda yer alan sınıfların türünün ne olduğunu belirten bir parametre tanımlanmasını sağlarlar.

### İççe (nested) sınıflar

- Sınıf içinde sınıf tanımı.
- UML bu durumları tanımlamak için bir özellik sunar.

### Kompozit Nesneler

- İçinde başka nesneler barındıran nesneler

## İleri İlişkiler/Bağlantılar

---

İsimlerle sınıfları, fiillerle işlemleri ve bağlantıları bulduk

### İşbirliği (Association)

- İnsan, araç → insanlar araçları kullanır.

### Genelleme (Generalization)

- Araba bir araçtır

### Bağımlılık (Dependency)

- Arabalar ve Emisyon Kanunları
- Emisyon kuralı değişirse, arabaların yeni bir cihaz takılması gibi adapte olmaları gerekir

## İleri ilişkiler

### Poligon **içerir** noktalar

- İfadenin sonundaki üçgen okuma yönünü gösterir
- İlişkilerin erişim yönlerini çizginin ucunu ok yaparak gösterebilirsiniz (navigability)

### İkinci ilişkinin adı yok

- Role göre koyabiliriz.
- İlişki sınıf da tanımlayabiliriz

### İçerme ilişkileri

- Açık dörtgen birleşme (aggregation)
- Poligon noktalardan yapılmıştır
- Kapalı dörtgen bütünleşme (composition)

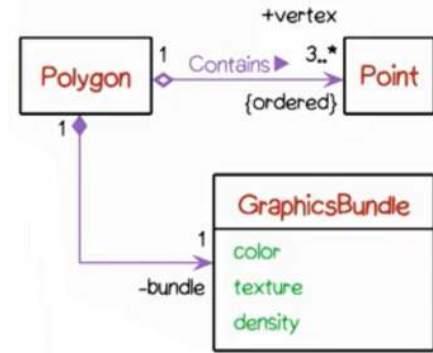
Çoğullamaları tanımlarız. Ör: 3..\*

### Kısıtlamaları tanımlarız.

- {ordered} stereotipi ile belirli bir sırada olmalarını belirtiyoruz.

Rolleri tanımlarız. Ör: bundle: bu ilişkide Grafik sınıfının rolü "bundle"

- Her iki uçta da tanımlayabiliriz.



## İlişki Sınıfları

Sınıf nitelikleri taşıyan ilişkileri ayrı bir sınıf olarak modelleyebiliriz.

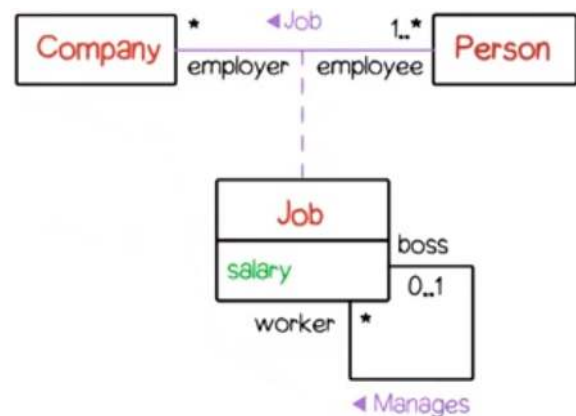
Kişi **çalışır** şirkette ilişkisinde, maaş özelliği tanımlayabiliriz.

- Maaş kişinin değil, çalışma ilişkisinin özelliğidir.
- Maaş şirketin özelliği de olamaz, pek çok çalışsını var.
- Kişinin birden fazla görevi için ayrı maaşları olabilir.

İlişki sınıfları, ilişki çizgisine dayanan kesikli çizgilerle gösterilir.

Burada İş sınıfının kendine ilişkisi vardır. Ozyinelemeli ilişki adı verilir.

- Rol tanımlarını iki uçta da yapmak faydalıdır.
- Patron **yönetir** işçiyi.



## Birleşme ve Bütünleşme

Birleşme: Bir sınıfın bir örneği, diğer sınıfın pek çok örneğinden yapılmıştır.

Yine bir ilişkidir. Birleşmeyi göstermek için içi boş dörtgen kullanılır.

Birleşme ilişkisi, ilişkinin anlamı ile ilgili bilgi vermez. Örneğin nenelerin yaşam ömürleri hakkında bilgi vermez.

Ev ve oda sınıflarımız olsun.

- Evin odaları vardır, burada birleşme ilişkisi vardır.
- Ancak, evi yıkarsak, tüm odaları da yıkmış oluruz.
- Buna bütünleşme ilişkisi diyoruz ve içi dolu dörtgen ile ifade ediyoruz.

Bütünleşmede, belirli bir öge sadece tek bir bütünleşmede yer alır.

- Ögenin yaşam süresini yönetme sorumluluğu da vardır.

Bütünleşmeler genellikle geçişken yapıdadır. Evin odaları, odanın gömme dolapları, ...

Odada masa olsun. Bu bir birleşmedir. Odayı yıkmadan önce masayı alırız. Bu nesnelerin birbirine bağlı olmayan ayrı yaşam ömürleri vardır.

## Hangi ilişki?

Dersler ve Öğrenciler

- Birleşme
- Öğrencileri yok etmeden dersi kapatabilirsiniz

Gelin ve Damat

- İlişki
- Bağımsız yaşam ömürleri var

Banka Hesabı ve Mudi

- Birleşme
- Mudinin banka hesabı olabilir

Yazı tipleri ve Gölgeler

- Bütünleşme
- Yazı tipi giderse tüm süslemeleri de gider

## Niteleyiciler (Qualifiers)

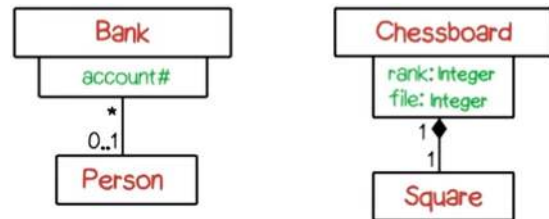
Sınıf dörtgenlerinin kenarında yer alan küçük dörtgenlerdir.

Sınıfın belirli bir özelliğinin ismini taşır.

- Bu özellik sınıfın nesnelere erişim sağlar
- Veritabanındaki birincil anahtar gibi düşünebiliriz.

Ör: insanlar bankaya hesap numaraları ile erişir

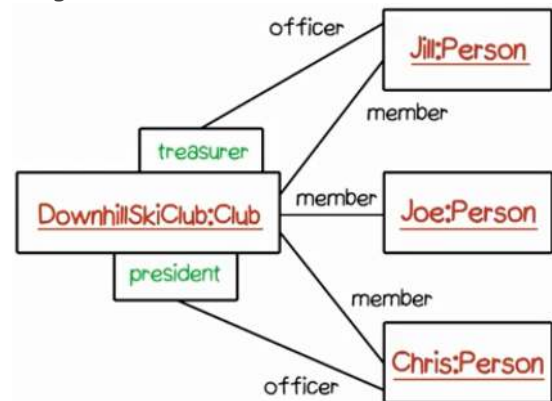
Ör: Satranç tahtasında kareler koordinat ile tanımlanır. (Burada bütünleşme de var)



## Bağlantılar (Link)

Sınıfların örnekleri olabildiği gibi, ilişkilerin de bağlantıları olabilir.

Bir nesneyi tekrar göstermek yerine her rol için bir bağlantı çizilir.





## Genelleme (Generalization)

Bağlantının ucunda üçgen yer alır.

Üçgen tarafındaki üst sınıf / ana sınıftır.

Diğer uçtaki(ler) alt sınıf / çocuk sınıftır.

Anlamı: alt sınıfın örnekleri aynı zamanda üst sınıfın da örneğidir.

Nesneye dayalı programlamadaki kalıtım ile birebir aynı kavram değildir!

- Kalıtım gerçekleştirme tekniğidir, genelleme is modellemedir.

UML'e çoklu üst sınıf ve çoklu alt sınıflar tanımlanabilir.

Ayrıştırıcılar (discriminator) ile alt sınıflar gruplanabilir.

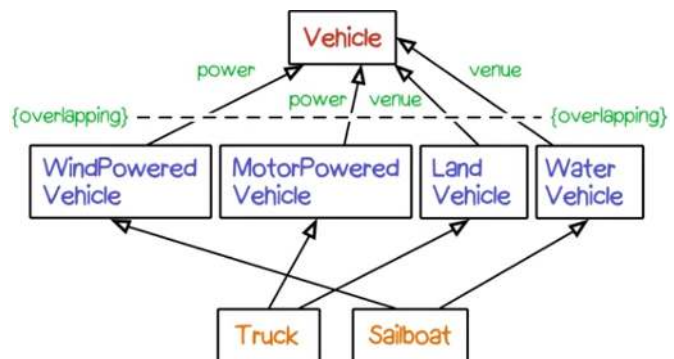
## Genelleme (Generalization)

Bir üst sınıftan türeyen alt alt sınıflar birden fazla alt sınıfa aitse {**overlapping**} kullanılır.

Sadece tek bir üst sınıfa ait olursa alt alt sınıflar {**disjoint**}dir

Çocuk sınıfın durumu, kendisinden türeyen tüm nesneleri kapsıyorsa **complete**, kapsamıyorsa **incomplete**

- Örneğin hibrit bir aracınız var ve çocukların hiçbirine uymuyor.



## Genelleme ve Kısıtlarına örnekler

Atlet: VoleybolOyuncusu, BasketbolOyuncusu, FutbolOyuncusu

- Overlapping: Bir oyuncu hem voleybol hem futbol oynayabilir.
- Incomplete: Bunları oynamayan atletler de var

Kitaplar: KağıtCiltli, ÇizgiRoman, BezCiltli

- Disjoint: Tekil bir kitap aynı anda ikisi birden olamaz
- InComplete: Başka kitap türleri de var.

Omnivorlar mı üst sınıftır, Vejetaryenler mi?

- Vejetaryenler

## UML Diyagramlarının Hiyerarşisi

